

Architecture — Synchronous Ephemeral Messenger

Status: design draft **Model:** zero-knowledge relay, session-only messaging, no server-side persistence

1. Design stance

This is a **session**, not a mailbox. Two parties are online simultaneously or there is no conversation. That single constraint is the source of nearly every property below — it is not a limitation to be worked around, it is the mechanism.

	Store-and-forward (Signal/WhatsApp)	This design
Offline delivery	Yes — server queues	No — dropped by design
Server state	Queues, prekeys, registration	None; RAM only, evaporates
Key exchange	Prekey bundles held by server	Live ephemeral DH, no server key material
Forward secrecy	Requires ratchet machinery	Falls out of the design almost free
History	Device-local, long-lived	Device-local, key-shredded on timer
Seizure of server yields	Metadata, queued ciphertext, prekeys	A process with no disk footprint

What it is not: a WhatsApp replacement. Those markets are taken and the offline-delivery requirement is precisely what forces their server-side complexity. Competing there means inheriting it.

2. Threat model

State this explicitly and design to it. Everything below is scoped by this table.

In scope

Adversary	Capability	Mitigation
Passive network observer	Reads all traffic	E2E encryption; Tor transport
Malicious/compromised relay	Full control of relay process	Zero-knowledge relay; authenticated DH so relay cannot MITM
Relay operator under compulsion	Subpoena, seizure, logging order	Nothing persisted to seize; ephemeral rooms
Attacker with a leaked room ID	Knows the room UUID	Room capacity 2; Tor client auth; crypto membership $\neq$ relay membership
Later device seizure	Full device access, post-hoc	Crypto-shredding; forward secrecy protects past sessions

Out of scope (say so out loud)

- Endpoint compromise **during** a live session — malware on either device sees plaintext, and no protocol fixes that.
- A global passive adversary correlating Tor entry/exit timing.
- The out-of-band channel used to deliver the initial invite.
- Coercion of a participant.

The honest residual

E2E hides *content*, not *association*. Under the synchronous model the relay observes, with precision, that identity A and identity B were connected during a specific interval, from specific IPs. **Synchronous-only is worse than store-and-forward for timing correlation** — store-and-forward blurs it. This is the main reason Tor is not optional here (see [TOR-TRANSPORT.md](#)).

3. Layers

| UI SwiftUI. Delivered vs dropped is a |

	first-class, always-visible state.	
Persistence	SQLCipher + per-conversation key.	
	Burn = destroy key, not rows.	
Session	Authenticated ephemeral X25519 →	
	HKDF → symmetric ratchet.	
Identity	Device keypair. userId derived from	
	public key. Secure Enclave if P-256.	
Transport	WebSocket over SOCKS5 over Tor.	
Relay	Opaque byte forwarding. RAM only.	

The relay sits *below* the security boundary. It is infrastructure, not a participant.

4. Identity layer

4.1 A UUID is an identifier, not a credential

The original sketch had clients assert `userId` and rooms be joinable by anyone holding the UUID. The relay could not test either claim, so presence, join events and delivery signals were all forgeable. Fix: **derive the UUID from a public key** and prove possession at connect.

identityKey ← generated once per device install

userId = uuid_v8(SHA-256(publicKey)[0..16])

The UUID mental model survives intact. It is now unforgeable.

4.2 Key algorithm — a real iOS constraint

The Secure Enclave supports P-256 only. It cannot hold Ed25519 or X25519 keys. CryptoKit exposes `Curve25519.Signing`, but those keys live in normal memory and are extractable from a compromised process.

	P-256 (<code>SecureEnclave.P256.Signing</code>)	Ed25519 (<code>Curve25519.Signing</code>)
Hardware-backed	Yes — key never leaves the SEP	No — software, extractable
Extractable by malware	No	Yes
Biometric gating	Yes, via access control	No
Speed, key size	Slower, larger sigs	Faster, 64-byte sigs
Cross-platform	Universal	Universal

Recommendation: P-256 in the Secure Enclave for the long-term identity key. For a security-focused product, a non-extractable identity key is worth more than signature size. The relay accepts both (`alg` field) so this stays reversible.

Watch the signature encoding: CryptoKit's `rawRepresentation` is fixed-width `r || s`; Node needs `dsaEncoding: "ieee-p1363"` to verify it. DER is the default and will fail silently-ish.

4.3 Ephemeral keys are deliberately software-only

Session keys use `Curve25519.KeyAgreement` in normal memory. This is correct, not a compromise: **forward secrecy requires that the key be destroyable**. A key you cannot delete is a key that can be compelled. Ephemeral keys must be cheap to create and trivially erasable — the Secure Enclave is the wrong home for them.

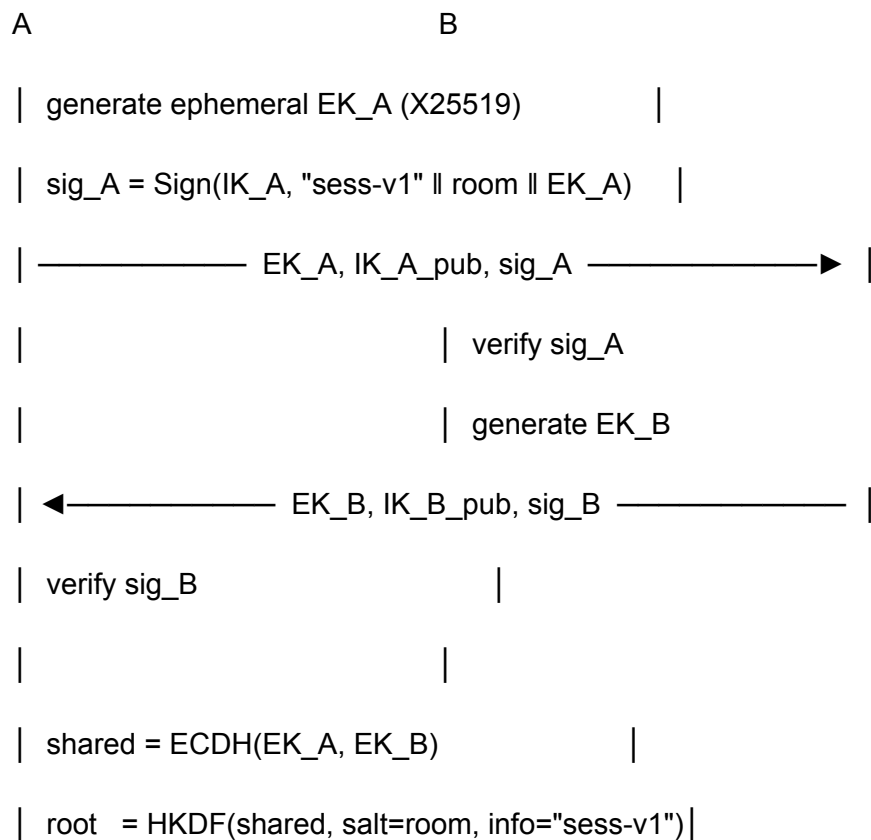
4.4 One device = one identity

A second device is a different `userId`. This is the simplest model and matches what most E2E systems land on. Multi-device requires either key sync (a whole subsystem) or treating a "user" as a set of devices with fan-out. **Defer, but decide consciously** — it is invasive to retrofit.

5. Session layer

5.1 Handshake

Both parties are online, so no prekey infrastructure is needed. This is X3DH with the offline half deleted.



Signing the ephemeral key with the identity key is what stops a malicious relay from substituting its own keys. Without it the relay MITMs the session trivially.

Deniability tradeoff: signatures are non-repudiable — they are cryptographic proof A participated. Signal avoids this by binding identity through DH instead of signatures. If deniability

matters for your users, use a triple-DH construction ($IK_A \times EK_B \parallel EK_A \times IK_B \parallel EK_A \times EK_B$) instead. That requires a P-256 *key agreement* key alongside the signing key, since one Secure Enclave key cannot do both. **Open decision — flag it.**

5.2 Message keys

Advance a chain key per message so each message key is deleted after use:

```
messageKey_n = HKDF(chainKey_n, info="msg")
```

```
chainKey_n+1 = HKDF(chainKey_n, info="chain")
```

Separate chains per direction. A full Double Ratchet is overkill for a single synchronous session — its purpose is recovering security across long asynchronous gaps, which you do not have.

5.3 Ordering and tamper-evidence

The relay assigns no sequence numbers (see §7). Ordering lives **inside** the ciphertext:

```
{ "n": 42, "prev": "<hash of message 41>", "ts": 1699..., "body": "..." }
```

The chain of **prev** hashes makes relay-side dropping or reordering detectable by the client without the relay's cooperation. Cheap, and it removes the last reason for the relay to understand message structure.

5.4 Authenticating first contact

There is no server key directory, so nothing vouches for IK_B on first contact. Derive a **safety number** from both identity keys, displayed as digits or a QR code, and verify out of band. Since the room ID already travels out of band, ship the fingerprint alongside it in the invite.

6. Transport layer — the wire protocol

WebSocket, JSON control frames, base64 opaque payloads. All frames use **t** for type.

6.1 Handshake

Dir	Frame	Notes
S→C	<pre>{"t": "challenge", "v": 1, "nonce": "<b64>"}</pre>	32 random bytes, per connection

Dir	Frame	Notes
C→S	<code>{"t":"auth","alg":"P-256","pk":"<b64>","sig":"<b64>"}</code>	signs "securechat-relay-auth-v1 " + nonce
S→C	<code>{"t":"ready","userId":"<uuid>","capacity":2}</code>	relay derives userId; client cannot choose it
S→C	<code>{"t":"error","code":"auth_failed",...} + close</code>	deliberately uninformative

Domain separation (`securechat-relay-auth-v1|`) prevents a signature captured here from being replayed into another protocol context. The nonce is single-use and per-connection, so replay across connections fails.

6.2 Session frames

Dir	Frame	Notes
C→S	<code>{"t":"open","room":"<uuid>"}</code>	creates room if absent
S→C	<code>{"t":"room","room":"<uuid>","peers":["<uuid>"]}</code>	peers already present
S→C	<code>{"t":"peer-join","room":...,"userId":...}</code>	only to authenticated room members
S→C	<code>{"t":"peer-leave","room":...,"userId":...}</code>	
C→S	<code>{"t":"msg","room":...,"payload":"<opaque>","ref":"<local id>"}</code>	payload never parsed
S→C	<code>{"t":"msg","room":...,"from":"<uuid>","payload":"<verbatim>"}</code>	
S→C	<code>{"t":"ack","ref":"<local id>","delivered":1}</code>	delivered:0 means dropped

Dir	Frame	Notes
C→S	<code>{"t":"close","room":...}</code>	

6.3 Full session

A: connect —————▶ relay

A: ◀— challenge(nonce)

A: — auth(pk, sig) —▶ relay verifies, derives userId_A

A: ◀— ready(userId_A)

A: — open(room) —▶ room created in RAM

A: ◀— room(peers: []) A waits; nothing can be sent yet

B connects, authenticates, opens same room

A: ◀— peer-join(userId_B) — room(peers:[A]) —▶ B

|

| == E2E handshake, relayed as opaque payloads ==

| A — msg(payload: EK_A||K_A||sig) —▶ relay —▶ B

| A ◀— msg(payload: EK_B||K_B||sig) ◀— relay ◀— B

| both derive root key

|

A: — msg(payload: ciphertext, ref:"local-7") —▶ relay —▶ B

A: ◀— ack(ref:"local-7", delivered:1)  shown as delivered

B backgrounds / loses signal

A: ◀— peer-leave(userId_B) UI: composer disabled, session over

A: — msg(...) —▶

A: ◀— ack(delivered:0) ✗ shown as DROPPED, never "sent"

6.4 Delivery semantics

delivered counts **peer sockets the frame was handed to** — not peer receipt, and certainly not decryption. It is a lower bound on failure, not a proof of success. True receipt is an application-level ACK inside the ciphertext.

This matters more here than in a normal chat app: **silent failure in a security tool is how people leak things**. A user who believes a message arrived will act on that belief. Never render an un-acked message in the same visual state as a delivered one, and never retry silently.

6.5 Reconnection

There is no resume. A dropped socket ends the session; the ephemeral keys are destroyed and a new session requires a fresh handshake. This is a feature — it is what makes forward secrecy per-session rather than per-lifetime. Reconnect should be explicit and user-visible, not automatic and silent.

7. The relay

Five invariants. If a change violates one, it is the wrong change.

1. **Nothing touches disk**. No logs, no queues, no metrics containing identifiers.
2. **Rooms are recounted by live sockets**. Last peer leaves → room deleted, no tombstone.
3. **payload is opaque**. Length-checked, forwarded verbatim, never parsed. The relay has no schema for message contents and must never acquire one, not even for debugging.
4. **No unauthenticated identity claims**. Signature over nonce, always.
5. **Capacity 2**. A third connection is refused. A relay that silently admits an extra peer is an eavesdropper.

Deliberately absent: sequence numbers, history buffers, typing indicators, read receipts, per-message compression (compressing attacker-influenced data alongside secrets is the CRIME shape).

The relay's peer list is a **routing hint, not the truth**. Real membership is whoever the session layer has agreed keys with. Never let the relay's view be authoritative about who is in a conversation.

8. Local persistence and burn

8.1 Deletion is the weakest link

A timer that deletes rows or unlinks a file **does not destroy data**. Flash wear levelling means overwrite-in-place is not something the application controls; the old blocks remain until the controller reuses them, and forensic tooling recovers them.

Use crypto-shredding. Destroy the key, not the data.

SecureEnclaveKey (P-256, non-extractable, biometric-gated)

└ wraps → ConversationKey (AES-256, per conversation)

└ encrypts → message rows in SQLCipher

Burn = delete **ConversationKey** from the Keychain. The ciphertext becomes indistinguishable from noise instantly and irreversibly, wherever its blocks physically live. Then delete the rows as housekeeping, not as the security mechanism.

Per-conversation keys also give per-conversation timers for free, which is what the user-controlled burn feature needs.

8.2 iOS leak paths — audit checklist

Every one of these has leaked plaintext from an otherwise-correct E2E app:

Path	Mitigation
Notification previews	Generic text only. Never put message content in the payload.
App-switcher snapshot	Cover the window in <code>sceneWillResignActive</code> , not <code>didEnterBackground</code> — too late.
iCloud / iTunes backup	<code>kSecAttrAccessibleWhenUnlockedThisDeviceOnly</code> , <code>.isExcludedFromBackup</code> on the DB.

Path	Mitigation
Keyboard learned words	<code>autocorrectionType = .no</code> , <code>spellCheckingType = .no</code> . Third-party keyboards see every keystroke — consider blocking them.
Pasteboard	<code>UIPasteboard</code> is system-wide and syncs via Handoff. Mark items <code>.localOnly</code> with an expiry, or disable copy.
Screenshots	Cannot be prevented. <code>userDidTakeScreenshotNotification</code> lets you at least notify the peer.
Crash logs / diagnostics	Never log message content, room IDs, or user IDs. Disable third-party crash SDKs entirely.
Memory	Swift <code>String</code> is not zeroable. Keep plaintext in <code>[UInt8]</code> you can wipe; keep its lifetime short.
Files app / Quick Look	Don't expose a document directory.

8.3 Data at rest when locked

Use `kSecAttrAccessibleWhenUnlockedThisDeviceOnly` and file protection `.complete`. Consequence: **the app cannot decrypt anything while the device is locked** — which is correct for this threat model, and another reason foreground-only operation fits.

9. iOS module layout

Identity/

IdentityStore Secure Enclave keygen, signing, userId derivation

SafetyNumber fingerprint rendering + QR verification

Session/

Handshake ephemeral DH, signature verification

Ratchet chain keys, per-message key derivation, key erasure

MessageCodec inner envelope: n, prev-hash, ts, body, padding

Transport/

TorController bootstrap, lifecycle, circuit state

RelayConnection WebSocket over SOCKS5; auth handshake; frame codec

ConnectionState one enum, single source of truth for the UI

Storage/

ConversationStore SQLCipher, per-conversation key

BurnScheduler timers, key destruction

UI/

...

Concurrency notes

- **RelayConnection** should be an **actor**. Its state machine has genuine interleaving hazards — an **open** racing a **peer-leave** racing a socket close is the exact shape that produces messages sent into a dead room.
 - **Beware actor reentrancy across **await****. Re-check state after every suspension point in the connection state machine; the peer may have left while you were awaiting the Tor circuit.
 - Ratchet state must not be an actor with **async** advance — a suspension point mid-ratchet lets two sends derive the same message key. Keep advancement synchronous and non-reentrant.
 - Cancellation must propagate to key destruction. A cancelled session that leaves ephemeral keys in memory has silently lost forward secrecy.
-

10. Open decisions

1. **Outbox semantics.** "Enqueue for when the peer returns" — *local outbox only*. If that queue ever lives on the relay you have rebuilt the mailbox and lost the entire design. Recommend: no queue at all; require an explicit resend.
 2. **Deniability** — signatures vs triple-DH (§5.2). Affects the Secure Enclave key layout, so decide before shipping identity.
 3. **Multi-device** — accept one-device-one-identity, or plan for it now.
 4. **Rendezvous.** How does a peer learn to come online? This is the hardest unsolved problem in the design; see [TOR-TRANSPORT.md §10](#).
 5. **Typing indicators — recommend removing entirely.** They leak keystroke timing, which is a well-studied side channel for inferring content, and they are noisy over Tor anyway. A security-focused app should not ship them.
-

11. Phasing

Phase	Scope	Purpose
0	Relay + signed auth + ephemeral rooms, ws://localhost	Protocol shape, client state machine
1	Ephemeral DH, ratchet, safety numbers	Real E2E; verify against a second simulator
2	SQLCipher, crypto-shredding, burn timers, leak-path audit	Data at rest
3	Tor: client → onion relay	Removes client IP from relay's view
4	P2P onion, relay reduced to rendezvous or eliminated	Removes the relay from the trust graph

Build 0–2 over plain WebSocket. Tor changes the transport, not the protocol — layering it in late is cheap, and debugging a handshake through a 6-hop circuit is not something to do while the handshake is still wrong.